

Universidad Nacional Abierta y a Distancia

Vicerrectoría Académica y de Investigación

Unidad gestora: *Escuela de Ciencias Básicas Tecnología e Ingeniería ECBTI*

Programa: *Ingeniería de Sistemas*

Curso: Programación

Código: 213023

Propuestas para el desarrollo de la Fase 2

En el presente documento se presentan cinco (5) propuestas de proyecto que serán desarrolladas en la Fase 2. El propósito de cada proyecto es permitir la aplicación práctica de los conocimientos adquiridos durante el curso.

Cada estudiante deberá seleccionar un (1) único proyecto para su desarrollo. El proyecto debe implementarse utilizando el lenguaje de programación Python y la biblioteca estándar Tkinter. Además, todas las interfaces gráficas deben estar diseñadas en idioma inglés.

Antes de acceder a cualquiera de los sistemas descritos en los ejercicios, el estudiante debe implementar un módulo de inicio de sesión (login) que valide un usuario que es programación y una contraseña que es programación.

Para esto, debe crearse una clase llamada **Usuario**, con:

- Atributos privados: `_usuario` y `_password`.

- Un método público validar(usuario_ingresado, password_ingresada) que retorne True únicamente si las credenciales coinciden.

La aplicación debe iniciar mostrando la pantalla de login en Tkinter.

Solo si el usuario ingresa credenciales válidas, se desbloquea el acceso al sistema principal de cada ejercicio.

Si las credenciales son incorrectas, el programa debe impedir el acceso y mostrar mensajes de error adecuados.

Ejercicio 1: Sistema de Control de Bicicletas en un Taller

Un taller de bicicletas requiere un sistema que permita registrar el ingreso de bicicletas para mantenimiento y calcular el costo total del servicio según el tiempo transcurrido.

El estudiante debe desarrollar una aplicación que gestione bicicletas usando Programación Orientada a Objetos y encapsulación.

Crear la clase **BicicletaTaller**, con:

- Atributos privados: _serial, _hora_ingreso, _costo_por_hora.
- Método registrar_ingreso(hora) para almacenar la hora de ingreso.
- Método registrar_salida(hora) para registrar la salida.
- Método calcular_total(hora_salida) para calcular el costo final.
- Método obtener_serial() para identificar la bicicleta.

El sistema debe:

- Permitir registrar bicicletas con sus datos.
- Guardarlas en una lista interna.

- Permitir seleccionar una bicicleta para registrar su salida.
- Calcular automáticamente el costo.
- Validar que la hora de salida sea mayor a la de ingreso.

Ejercicio 2: Control de Reservas de Salas de Estudio

Una biblioteca universitaria requiere gestionar las reservas de salas de estudio. Cada reserva debe tener un usuario responsable, una hora de inicio y una tarifa por hora.

Crear la clase **ReservaSala**, con:

- Atributos privados: `_usuario`, `_hora_inicio`, `_tarifa_hora`.
- Método `registrar_inicio(hora)`
- Método `registrar_fin(hora)`
- Método `calcular_costo(hora_fin)`
- Método `obtener_usuario()`

El programa debe:

- Crear reservas y almacenarlas en una lista.
- Permitir seleccionar una reserva y registrar la hora de fin.
- Calcular automáticamente el costo final.
- Validar horas incorrectas o vacías.

Ejercicio 3: Sistema para Control de Lavado

Un autolavado desea controlar los autos que ingresan al servicio de lavado y calcular el costo según la cantidad de tiempo que permanezcan en el proceso.

Crear la clase **AutoLavado**, con:

- Atributos privados: `_placa`, `_hora_ingreso`, `_tarifa_hora`.
- Método `registrar_ingreso(hora)`
- Método `registrar_salida(hora)`
- Método `calcular_pago(hora_salida)`
- Método `obtener_placa()`

El sistema debe:

- Registrar autos en una lista interna.
- Permitir seleccionar un auto para registrar la salida.
- Calcular el costo automáticamente.
- Validar la hora de salida.

Ejercicio 4: Control de Computadores en Mantenimiento

Un centro de soporte técnico necesita controlar los computadores que ingresan a revisión y calcular el valor del diagnóstico según el tiempo.

Crear la clase **ComputadorMantenimiento**, con:

- Atributos privados: `_codigo`, `_hora_entrada`, `_valor_hora`.
- Método `registrar_entrada(hora)`

- Método registrar_salida(hora)
- Método calcular_valor(hora_salida)
- Método obtener_codigo()

La aplicación debe:

- Registrar computadores en mantenimiento.
- Guardarlos en una lista interna.
- Seleccionar un computador para registrar su salida.
- Calcular y mostrar el costo total.
- Validar correctamente las horas.

Ejercicio 5: Sistema de Alquiler de Herramientas

Una tienda de alquiler de herramientas desea registrar el tiempo de alquiler de cada herramienta y calcular el costo final según las horas usadas.

Crear la clase **HerramientaAlquiler**, con:

- Atributos privados: _id_herramienta, _hora_salida, _tarifa_hora.
- Método registrar_salida(hora)
- Método registrar_retorno(hora)
- Método calcular_costo(hora_retorno)
- Método obtener_id()

El sistema debe:

- Registrar herramientas en una lista interna.

- Permitir registrar retorno.
- Calcular automáticamente el costo del alquiler.
- Validar horas incorrectas.